

Design of an AI-Driven Hostel Allocation and Complaint Management System (AHCMS)

Ritesh Roshan Sahoo

April 6, 2026

Abstract

This document presents a technical proposal for upgrading the AHCMS (Amrita Hostel & Complaint Management System) into a large-scale, production-ready platform. The report analyzes the current project structure, formulates the hostel allocation and complaint-routing problems, evaluates candidate optimization algorithms recommended for the project, and justifies the most suitable algorithmic and architectural choices for implementation. The proposed direction combines constraint-based optimization for room allocation, rule-based and priority-driven complaint routing, real-time communication infrastructure, and a scalable service-oriented backend, with explicit attention to fairness, ethics, and psychological aspects of hostel life.[1, 2, 3, 5, 6]

1 Introduction

The current AHCMS project is a web-based hostel and complaint management platform intended to support student accommodation workflows and complaint handling in a university hostel environment. The public repository contains a frontend source directory, a backend `server` directory, and an `sql` directory, which indicates that the system already includes both application logic and relational database support.[1] The deployed application presents login flows for students and administrators and describes itself as “Amrita Hostel & Complaint Management, Delhi NCR”, showing that the project already functions as an integrated hostel operations portal.[2]

The next stage of the project, based on faculty recommendations, is not merely feature addition but system maturation. The required upgrades include automated room and resource allocation, intelligent prioritization, hierarchical complaint routing, Discord-based communication, push notifications, and scalable architecture for high-concurrency production use. These requirements naturally lead to a combination of database design, scheduling theory, optimization algorithms, and distributed systems engineering.[2, 3, 5]

2 Current System Assessment

The GitHub repository shows that the project is primarily implemented in JavaScript, with supporting CSS and HTML, and that it includes distinct directories for backend logic, SQL scripts, and frontend application code.[1] This structure is suitable for incremental migration

toward a service-oriented design because the main business capabilities are already separated at a coarse level into application, data, and interface layers. The commit history indicates active development around room allocation management and user experience improvements.[1]

The deployed application currently exposes a unified student/admin portal with hostel-focused branding and demonstrates that the system is already intended to support room allocation and complaint management together.[2] From an engineering perspective, this means the project does not need a conceptual redesign from scratch; instead, it needs formalization of constraints, robust database relationships, algorithmic automation, and architecture upgrades to support reliability, scale, and maintainability.

3 UID and DBMS Design

3.1 Unique Identifier Strategy

A production-grade hostel management system requires stable and unambiguous identifiers for students, rooms, complaints, staff, and events. The most practical choice is to retain the institutional roll number as the logical identifier for each student while using surrogate numeric keys for internal entities such as rooms, allocations, staff members, and complaint tickets. This approach preserves human readability where useful and ensures database efficiency, indexing consistency, and safe integration across services.[1, 2]

The following UID strategy is recommended:

- **Student UID:** Use the university roll number as a unique functional identifier and map it to an internal primary key when needed.
- **Room UID:** Use a numeric `room_id` as the primary key and a human-readable `room_code` such as B3-402.
- **Complaint UID:** Use a sequential ticket ID or compact UUID for easy referencing in dashboards, Discord, and notifications.
- **Staff UID:** Use a numeric `staff_id` with role metadata such as warden, electrician, plumber, or supervisor.
- **Event UID:** Use immutable event IDs for complaint status changes, escalations, and notification triggers.

3.2 Database Schema Recommendations

The presence of an `sql` directory in the repository confirms that the current project already relies on a relational data model.[1] To support advanced scheduling, complaint routing, and event-driven communication, the schema should be extended into a normalized design with clear foreign keys, indexed lookup fields, and auditable event history. A relational DBMS remains the right primary choice because the core entities—students, rooms, complaints, staff, assignments, and escalation rules—have strong structural relationships and transactional requirements.

A suitable schema extension is outlined below:

- **STUDENT**(student_id, roll_number, name, gender, batch, program, contact, status)
- **ROOM**(room_id, room_code, block, floor, capacity, room_type, gender_allowed, status)
- **ALLOCATION**(allocation_id, student_id, room_id, start_date, end_date, allocation_status)
- **COMPLAINT**(ticket_id, student_id, category_id, block, room_code, description, priority, status, created_at, updated_at, current_assignee)
- **COMPLAINT_CATEGORY**(category_id, name, default_role, sla_minutes)
- **STAFF**(staff_id, name, role, block_assigned, contact, discord_tag, availability_status)
- **ROUTING_RULE**(rule_id, category_id, block, role, escalation_role, sla_minutes)
- **TICKET_EVENT**(event_id, ticket_id, event_type, old_value, new_value, created_at, actor)
- **NOTIFICATION**(notification_id, user_id, channel, payload, delivery_status, created_at, sent_at)

This schema supports both transaction processing and analytics. It also creates a foundation for automation because room allocation, complaint routing, escalation, and notifications all depend on clean relational mappings rather than ad hoc logic scattered across frontend and backend code.[1, 2]

4 Scheduling and Allocation Problem Formulation

Room allocation in AHCMS can be modeled as a constrained optimization problem. Let $x_{ij} \in \{0, 1\}$ denote whether student i is assigned to room j . The problem contains hard constraints such as room capacity, gender compatibility, one-room-per-student assignment, and policy-specific block or batch restrictions. In addition, it includes soft objectives such as balanced occupancy, reduced student relocation across semesters, and grouping of preferred peers when allowed.[3, 4, 6]

Formally, the key hard constraints can be written as:

$$\sum_i x_{ij} \leq \text{capacity}(j) \quad \forall j$$

$$\sum_j x_{ij} \leq 1 \quad \forall i$$

These conditions guarantee that no room exceeds capacity and no student is assigned to more than one room. Additional binary or logical constraints can be introduced to encode gender rules, room type compatibility, accessibility needs, or hostel policy rules.[3, 4]

This formulation is closely related to assignment, bin-packing, and timetabling problems, all of which are computationally hard in general. However, modern combinatorial optimization solvers and carefully designed metaheuristics can solve institution-sized instances effectively in practice.[3, 5, 6]

5 Evaluation of Candidate Algorithms

5.1 Swarm Intelligence and PSO

Swarm Intelligence refers to optimization techniques inspired by collective behavior in nature, such as bird flocking, fish schooling, or ant foraging. Particle Swarm Optimization (PSO) is one of the most widely used swarm-based methods, where candidate solutions move through the search space based on both their own historical best position and the best-known position of the population. Hybrid GA–PSO approaches have been shown to improve workflow scheduling quality in cloud environments by balancing exploration and exploitation.[5]

For this project, PSO is attractive because it is conceptually aligned with faculty recommendations and can be adapted for discrete allocation problems. However, its main limitation in hostel allocation is constraint handling: the raw PSO framework is naturally better suited to continuous spaces, so discrete feasibility repair, penalty design, and parameter tuning become important and sometimes complex. As a result, PSO is better treated as a comparative or future-extension algorithm rather than the primary production solver.[5]

5.2 Genetic Algorithms

Genetic Algorithms (GAs) use a population of encoded candidate solutions and evolve them through selection, crossover, and mutation. They are especially popular in scheduling, timetabling, and multi-objective allocation problems because they can search a large solution space without requiring a rigid mathematical structure. Recent timetable-generation work based on GA and constraint satisfaction demonstrates that GAs remain highly relevant when the problem mixes hard constraints with preference-driven soft goals.[6]

For AHCMS, a GA can represent each candidate chromosome as a hostel allocation plan or as a complaint-task assignment order. Its main strength is flexibility: it can optimize multiple objectives and can generate several near-optimal alternatives rather than a single exact solution. Its main weakness is that pure GA implementations do not guarantee constraint satisfaction unless coupled with repair operators, penalty functions, or a secondary feasibility checker.[6]

5.3 SAT and CP-SAT Based Optimization

SAT-based optimization encodes constraints into Boolean or pseudo-Boolean formulations and uses highly optimized solvers to find feasible or optimal assignments. Google OR-Tools provides the CP-SAT solver, which is explicitly designed for integer and combinatorial optimization problems and is generally faster than the older MPSolver pathway for such models. OR-Tools also provides documented patterns for assignment and scheduling problems, which closely match hostel allocation requirements.[3, 4]

For AHCMS, CP-SAT is particularly strong because it handles binary decisions, hard constraints, and weighted objective functions directly. This means room capacities, gender restrictions, room eligibility, and policy constraints can be modeled exactly rather than approximated. In a system where infeasible assignments are unacceptable, this exact constraint handling gives CP-SAT a significant practical advantage over purely metaheuristic approaches.[3, 4]

5.4 Rule-Based AI and Priority Scheduling

Not every part of the system requires a heavy optimization engine. Complaint prioritization and staff queue ordering are often better served by interpretable rule-based logic such as earliest-deadline-first (EDF), severity scoring, and workload-aware assignment.[3] This style of automation is still a legitimate AI-driven system in the broad sense because it uses structured decision rules and prioritization policies to automate operational outcomes.

For AHCMS, this means the project should distinguish between two classes of automation: global combinatorial optimization for room allocation, and deterministic operational prioritization for complaint handling. Making this distinction improves both performance and explainability.[2, 3]

6 Best Algorithm Choice for AHCMS

The best primary algorithm for this project is **OR-Tools CP-SAT**, with an optional **Genetic Algorithm layer** for research extension or initial-solution generation. This recommendation is based on the structure of the problem, the requirement for strict hard constraints, and the need for a solution that is both academically defensible and implementable in a production-grade system. OR-Tools explicitly supports combinatorial optimization and assignment modeling, while GA provides a flexible secondary technique that matches faculty recommendations and can be used for comparative experimentation.[3, 4, 6]

The justification is:

1. **Feasibility is critical.** Room over-allocation, gender-rule violations, or duplicate assignments are unacceptable, and CP-SAT handles such hard constraints natively.[3, 4]
2. **The problem is naturally discrete.** Students are assigned or not assigned to rooms; this binary structure fits CP-SAT directly.[4]
3. **The solver is production-oriented.** OR-Tools is a mature optimization library with practical support for assignment and scheduling models.[3]
4. **GA remains useful.** A GA can be added later for multi-objective experimentation, preference exploration, or hybrid initialization, which gives the project additional academic depth without making the production system unstable.[6, 5]
5. **PSO is less suitable as the first implementation.** Although powerful in scheduling literature, PSO requires more careful adaptation for discrete hard-constrained hostel allocation than CP-SAT.[5]

Therefore, the final recommendation is: use CP-SAT as the main automated hostel allocation engine, use rule-based priority logic for complaint scheduling, and include GA as a hybrid or future comparative enhancement.[3, 4, 5, 6]

7 Fairness, Ethics, and Psychological Considerations

7.1 Motivation for Fair Allocation

Hostel room assignment is not only a combinatorial optimization problem but also a socio-technical system that affects students' sense of fairness, belonging, and well-being. Empirical work on dorm allocation and fair resource sharing shows that even when allocations are efficient, students frequently complain if they perceive the outcome as unfair relative to peers or if they feel their preferences and social ties were ignored.[7, 8] The same literature emphasizes that fairness should be treated as a first-class design goal, with explicit mathematical criteria and transparent communication, rather than an informal by-product of ad hoc rules.[7, 8, 11]

7.2 Fairness Metrics for Hostel Allocation

To move beyond purely utilitarian objectives, the AHCMS allocation engine incorporates fairness metrics drawn from fair-division and dorm-assignment research.[8, 9, 10] The main concepts are as follows.

7.2.1 Individual Envy-Freeness

Consider a set of students N and a set of rooms R , where each student $i \in N$ has a preference ranking or utility function u_i over possible room assignments (including dorm-mates, when relevant). An allocation X assigns each student to at most one room. The allocation is called *envy-free* (*EF*) if no student strictly prefers another student's assignment:

$$u_i(X_i) \geq u_i(X_j) \quad \forall i, j \in N.$$

In words, no student would rather have both the room and the dorm-mates of any other student.[7, 8]

Exact envy-freeness can be difficult to guarantee when rooms are indivisible and have capacity greater than one, so the system instead aims for approximate fairness.

7.2.2 Envy-Freeness up to One Room (EF1)

An allocation X is *envy-free up to one room* (*EF1*) if for any pair of students (i, j) , student i does not envy j after the hypothetical removal of at most one component of j 's bundle:

$$\exists \text{ component } c \text{ of } X_j \text{ such that } u_i(X_i) \geq u_i(X_j \setminus \{c\}).$$

EF1 is widely used in fair allocation of indivisible goods when exact envy-freeness is impossible; it captures the idea that any perceived advantage of another student's allocation is limited to at most one salient element.[9, 10] The AHCMS solver does not explicitly enforce EF1 as a hard constraint, but the GA layer and post-processing heuristics can search for allocations that minimize the number or severity of envy relations as measured by this notion.

7.2.3 Group Fairness Constraints

Beyond individual fairness, hostel allocation must avoid systematic disadvantage to identifiable groups, such as gender, academic program, or batch year. Let $G_1, G_2, \dots, G_k$ be a partition of the student population into groups of interest. For each student i , define a satisfaction score s_i based on their allocation (e.g., distance to preferred block, match with preference list, noise level, and compatibility).

For each group G_g , define the average satisfaction

$$\bar{s}(G_g) = \frac{1}{|G_g|} \sum_{i \in G_g} s_i.$$

The allocation engine introduces soft constraints or penalty terms that limit disparities between groups:

$$\max_{g,h} |\bar{s}(G_g) - \bar{s}(G_h)| \leq \Delta_{\max},$$

for a designer-chosen threshold $\Delta_{\max}$. [8, 11] The objective function then combines overall satisfaction with a penalty proportional to this maximum gap, encouraging solutions in which no demographic group is consistently assigned substantially worse rooms than others.

7.2.4 Max-Min Fairness

Max-min fairness focuses on protecting the least-advantaged student. Given satisfaction scores s_i , an allocation is more max-min fair if it maximizes the minimum satisfaction among all students:

$$\max_X \min_{i \in N} s_i(X).$$

In practice, CP-SAT can incorporate this idea by introducing an auxiliary variable z representing the minimum satisfaction and maximizing z subject to $z \leq s_i$ for all students. [9, 10] This helps prevent highly unequal allocations where a small subset of students receive excellent rooms while others receive very poor ones.

7.3 Fairness-Aware Objective Design

In the proposed model, the CP-SAT objective is a weighted combination of utility and fairness terms:

$$\max \alpha \sum_i s_i - \beta \sum_{(i,j) \in E} \text{envy}_{ij} - \gamma \max_{g,h} |\bar{s}(G_g) - \bar{s}(G_h)|,$$

where E is the set of student pairs with potential envy relations, envy_{ij} quantifies the degree to which i prefers j 's allocation, and α, β, γ are design parameters selected in consultation with administrators and student representatives. [8, 11] This construction makes fairness an explicit optimization target rather than an informal hope.

The optional GA layer is configured with a multi-objective fitness function that mirrors these terms, so that candidate solutions lying on a good utility–fairness Pareto frontier can be generated and then refined by CP-SAT. [8, 10]

7.4 Ethical and Psychological Considerations

7.4.1 Perceptions of Fairness and Trust

Research on algorithmic decision-making indicates that students care not only about the outcome of an allocation but also about the perceived fairness of the process and the transparency of the reasoning.[11, 12] When decisions are made by an opaque algorithm, individuals may perceive unfairness yet feel less able to contest it, which can erode trust and increase feelings of helplessness.

To address these concerns, the AHCMS design explicitly supports: (i) understandable explanations for each allocation; (ii) an appeals workflow where students can submit additional context or request review; and (iii) public, anonymized statistics on group-level outcomes so that systematic disparities are visible and can be corrected.[11, 12]

7.4.2 Social Dynamics in Shared Living

Fair dorm assignment is not only about matching students to physical rooms but also about managing interpersonal externalities among roommates and neighbours. Dorm-assignment models that allow students to care about both dorms and dorm-mates align with psychological evidence that peer compatibility, lifestyle similarity, and social support strongly influence perceived well-being in shared housing.[7, 8]

While the current AHCMS implementation focuses on room and block constraints, the framework can be extended to incorporate simple compatibility signals (such as preferred noise levels or sleep schedules) and to avoid obviously incompatible roommate pairings. These social factors can be reflected in the satisfaction scores s_i and thus in the fairness-aware objective, connecting the mathematical allocation with students’ lived psychological experience.[7, 11]

7.4.3 Algorithmic Accountability

Ethics in AI is fundamentally about dignity, justice, and accountability.[11, 12] The system should therefore maintain logs of allocation and complaint decisions, document model versions and parameter settings, and require human review for flagged disparities or exceptional cases. By combining rigorous optimization with governance mechanisms and user-facing transparency, AHCMS can be positioned as not only efficient but also responsible and psychologically considerate.

8 Automated Complaint Management

8.1 Hierarchical Routing

Complaint management in AHCMS should be modeled as a hierarchical routing and escalation system. Each complaint must carry structured metadata such as category, hostel block, room, priority, and time of submission. Based on these attributes, the system should automatically map the complaint to the most relevant staff role—for example, plumbing complaints to a

plumber, electrical complaints to an electrician, and unresolved escalations to a warden or supervisor.[2, 1]

This logic should be implemented using database-backed routing rules rather than hard-coded branching scattered across the application. A routing table allows the backend to query the appropriate assignee dynamically, which makes the system easier to maintain and extend when categories, blocks, or staff assignments change. This design also supports explainable automation because every ticket assignment can be traced back to a rule record.[1]

8.2 Prioritisation Strategy

Complaint prioritization should not rely on a complex evolutionary algorithm in the first production version. A more suitable model is a weighted operational priority score composed of severity, service-level deadline, escalation state, and local batching efficiency. In practice, this becomes a queue discipline similar to earliest-deadline-first, with tie-breaking based on complaint severity and proximity of tasks within the same block or floor.[3]

A simple scoring function can be used:

$$\text{PriorityScore} = w_1(\text{severity}) + w_2(\text{urgency}) + w_3(\text{escalation}) - w_4(\text{travel_cost}),$$

which is transparent, easy to tune, and operationally realistic.

9 Discord Bot and Push Notifications

A Discord bot is a strong fit for the project because it adds a lightweight real-time communication layer without requiring every staff member to remain inside the web dashboard. The bot can support commands for checking complaint status, listing assigned tickets, escalating unresolved issues, and broadcasting alerts to maintenance groups or hostel administrators. Since the deployed system already supports distinct user roles, the Discord bot should be implemented as a separate service that interacts with the complaint-management backend through authenticated APIs.[2, 1]

Push notifications should be event-driven rather than directly coupled to the main transaction request. Whenever a complaint is created, assigned, updated, or escalated, the system should publish an event that triggers notification delivery to the relevant student, staff member, or Discord channel. This design reduces coupling, improves responsiveness, and supports future channels such as email, SMS, or mobile notifications.[2]

10 Scalable Architecture

To evolve from a student project into a large-scale production-ready platform, AHCMS should adopt a modular service architecture. A reasonable decomposition includes an API gateway, an allocation service, a complaint service, a notification service, and a Discord bot service. The repository already shows backend and SQL separation, so this is a natural extension rather than a disruptive redesign.[1]

Containerization and asynchronous messaging are the most suitable next steps for scalability. In such an architecture, user-facing APIs remain responsive while long-running tasks such as bulk room allocation, notification dispatch, or escalated workflows are handled asynchronously. This approach matches the stated requirement for a scalable, event-driven, microservices-oriented system and is the correct direction if the platform is expected to support large hostel populations or multiple campuses.[2, 3]

11 Final Recommendation

For the AHCMS project, the most appropriate algorithmic strategy is not to use every recommended method equally, but to map each method to the part of the system where it fits best. The strongest production choice is to use **OR-Tools CP-SAT** for automated room allocation because it directly models binary decisions and strict constraints.[3, 4] For complaint handling, the best practical choice is **hierarchical rule-based routing with deadline-aware priority scoring**, because it is transparent, efficient, and easy to justify. A **Genetic Algorithm** can be retained as a hybrid enhancement or research comparison method, while **PSO and broader Swarm Intelligence methods** should be presented as future work rather than the first deployed solution.[5, 6]

This recommendation is a good fit for the faculty brief because it is technically rigorous, implementation-friendly, fairness-aware, and easy to defend academically. It shows that advanced AI and optimization methods were evaluated seriously, but that the final choice was driven by the structure of the actual problem and by ethical considerations rather than by fashion.

References

- [1] R. R. Sahoo. *riteshroshann/ahcms GitHub Repository*. GitHub. Available at: <https://github.com/riteshroshann/ahcms>.
- [2] AHCMS. *AHCMS — Amrita Hostel & Complaint Management*. Available at: <https://ahcms.vercel.app/>.
- [3] Google Developers. *CP-SAT solver | OR-Tools*. Available at: https://developers.google.com/optimization/cp/cp_solver.
- [4] Google Developers. *Assignment with task sizes | OR-Tools*. Available at: https://developers.google.com/optimization/assignment/assignment_cp.
- [5] A. M. Manasrah, A. Al-Badarneh, and S. Sababha. Workflow scheduling using hybrid GA-PSO algorithm in cloud computing environment. *Scientific Programming*, 2018.
- [6] S. Patel and K. Jani. Timetable generator using genetic algorithm and constraint satisfaction. *International Journal of Fundamental and Multidisciplinary Research*, 2025.
- [7] J. Gan, B. Li, and Y. Wu. Your college dorm and dormmates: Fair resource sharing in dorm assignment. *Journal of Artificial Intelligence Research*, 2023.

- [8] B. Li, J. Gan, and Y. Wu. Fair resource sharing and dorm assignment. In *Proceedings of the 19th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2020.
- [9] S. Barman, S. Krishnamurthy, and R. Vaish. Fair division under cardinality constraints. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI)*, 2018.
- [10] H. Aziz, B. Li, H. Moulin, and X. Wu. Algorithmic fair allocation of indivisible items: A survey and new questions. *SIGecom Exchanges*, 20(1):24–40, 2022.
- [11] A. Yapo and J. Weiss. Ethical implications of bias in machine learning. In *Proceedings of the 51st Hawaii International Conference on System Sciences*, 2018.
- [12] Crimson Publishers. Bias, fairness, and ethical accountability in artificial intelligence. *COJRA Journal of Robotics & Automation*, 2026.